

Struttura dell'applicazione

ICM Fatturazioni

Blazor Web App · .NET 10 · render mode `InteractiveServer`

Lo scaffolding iniziale e la catena dei file: da `Program.cs` alla pagina Anagrafiche.

Architettura layered: UI (Blazor) → Manager → Repository (Dapper) → SQL Server. Documento generato il 10/06/2026.

1. Il quadro d'insieme: due momenti distinti

Per capire come i file si chiamano in sequenza bisogna tenere separati **due momenti** che spesso vengono confusi:

A · L'avvio dell'applicazione (una volta sola)

Quando lanci `dotnet run`, viene eseguito **Program.cs** dall'alto verso il basso: **registra i servizi** (DI), **costruisce la pipeline HTTP**, mappa gli endpoint e fa il seed dell'utente di sviluppo. Da qui in poi il server resta in ascolto.

B · La singola richiesta dell'utente (ogni volta)

Quando il browser chiede una pagina, la richiesta attraversa la pipeline costruita in A, arriva a **App.razor** (il documento HTML radice), che ospita **Routes.razor** (il router), che sceglie la pagina e la avvolge nel **MainLayout**. La pagina (**Anagrafiche.razor**) parla coi Manager, che parlano coi Repository.

La catena dei file in una riga

1 **Program.cs** — registra servizi + pipeline, poi `MapRazorComponents<App>()`



2 **App.razor** — documento HTML radice (`<head>`, CSS, script Blazor) → ospita `<Routes>`



3 **Routes.razor** — il `<Router>` abbina l'URL alla pagina e applica il layout di default



4 **MainLayout.razor** — cornice (AppBar, Drawer, provider MudBlazor) + `@Body`



5 **Anagrafiche.razor** — la pagina; nel `@Body` → chiama **IAnagraficaManager**



6 **AnagraficaFormDialog.razor** — il dialog Aggiungi/Modifica, aperto su richiesta dalla pagina

2. Program.cs — il compositore

È il punto d'ingresso. Si legge in **due metà**: prima si *registrano* i servizi nel container DI (`builder.Services...`), poi — dopo `builder.Build()` — si *configura* la pipeline HTTP e si mappano gli endpoint. L'ordine della seconda metà è vincolante.

2.1 Registrazione dei servizi (DI)

```
// Blazor Web App in modalità Server interattivo
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
builder.Services.AddMudServices(); // Snackbar, Dialog, Theme, Breakpoint...

// Cookie auth + policy che richiede login su TUTTA l'app
builder.Services.AddAuthentication(...).AddCookie(...);
builder.Services.AddAuthorizationBuilder()
    .SetFallbackPolicy(new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build());
builder.Services.AddCascadingAuthenticationState();

// Catena dati: factory → repository → manager (ciclo di vita scelto con cura)
builder.Services.AddSingleton<ISqlConnectionFactory, SqlConnectionFactory>();
builder.Services.AddScoped<IAnagraficaRepository, AnagraficaRepository>();
builder.Services.AddScoped<IAnagraficaManager, AnagraficaManager>();
builder.Services.AddSingleton<ILogger, ILogger>();
builder.Services.AddScoped<CircuitHandler, IcmCircuitHandler>();
```

Servizio	Ciclo di vita	Perché
ISqlConnectionFactory	Singleton	Stateless: crea una <code>SqlConnection</code> nuova ad ogni chiamata, nessuno stato condiviso.
IAnagraficaRepository / Manager	Scoped	Una istanza per request/circuit, coerente con la durata della connessione.
ILogger	Singleton	Canale di logging "infallibile" usato da middleware, circuit handler, ErrorBoundary e Manager.

2.2 La pipeline HTTP (l'ordine conta)

```
app.UseExceptionHandler(); // → GlobalExceptionHandler logga e poi lascia la dev/Error page
app.UseStatusCodePagesWithReExecute("/not-found");
app.UseHttpsRedirection();
app.UseAuthentication(); // 1° CHI sei (legge il cookie)
app.UseAuthorization(); // 2° COSA puoi fare (fallback policy)
app.UseAntiforgery();
app.MapStaticAssets();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode(); // ← entra App.razor
```

Punto chiave: `MapRazorComponents<App>()` dice ad ASP.NET «il componente radice di tutta l'UI è `App`». È il ponte tra il mondo HTTP e il mondo Blazor.

2.3 Endpoint di login e seed

Perché il login è un endpoint e non un componente

In Blazor Server interattivo non puoi chiamare `SignInAsync` da un componente: la response è già "agganciata" al circuit. Per questo il form di `/login` fa un **POST HTML classico** a `/auth/login` (minimal API in **Program.cs**), dove l' `HttpContext` è ancora scrivibile: lì si validano le credenziali, si creano i *claim* e si scrive il cookie.

In Development viene anche eseguito `SeedUtenteSviluppoAsync()` (utente admin / admin1234), protetto da try/catch che logga via `IErrorLogger` se il DB non è ancora migrato.

3. App.razor — il documento HTML radice

È letteralmente la pagina `<html>` che il browser riceve. Carica i CSS (MudBlazor, `app.css`, gli stili scoped), i font, lo script `blazor.web.js` e — soprattutto — ospita il router.

```
<head>
  <link rel="stylesheet" href="_content/MudBlazor/MudBlazor.min.css" />
  <link rel="stylesheet" href="@Assets["app.css"]" />
  <HeadOutlet />           // permette alle pagine di iniettare <PageTitle>
</head>
<body>
  <Routes @rendermode="InteractiveServer" />  // ← il render mode è applicato QUI, globale
  <ReconnectModal />
  <script src="_content/MudBlazor/MudBlazor.min.js"></script>
  <script src="@Assets["_framework/blazor.web.js"]"></script>
</body>
```

Due dettagli che spiegano molto

- `@rendermode="InteractiveServer"` su `<Routes>`: rende interattiva tutta l'app. È ciò che permette ai provider MudBlazor (Dialog, Snackbar...) nel layout di funzionare.
- `<HeadOutlet />`: è il "buco" dove finiscono i `<PageTitle>` dichiarati nelle pagine (es. "Anagrafiche · ICM Fatturazioni").

4. Routes.razor — il router

```
<Router AppAssembly="typeof(Program).Assembly" NotFoundPage="typeof(Pages.NotFound)">
  <Found Context="routeData">
    <RouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)" />
    <FocusOnNavigate RouteData="routeData" Selector="h1" />
  </Found>
</Router>
```

Compito unico: **guarda l'URL e trova la pagina** con l'attributo `@page` corrispondente (es. `@page "/anagrafiche"` → **Anagrafiche.razor**). Tre cose da notare:

- **AppAssembly**: cerca le pagine in tutto l'assembly dell'app.
- **DefaultLayout**: ogni pagina trovata viene avvolta nel `MainLayout` (a meno che non dichiari un layout proprio).
- **FocusOnNavigate ... Selector="h1"**: dopo ogni navigazione sposta il focus sull'`<h1>` (accessibilità).

5. MainLayout.razor + NavMenu — la cornice

Il layout è il "telaio" comune a tutte le pagine. Dichiara i **provider MudBlazor** (una volta sola, qui), la barra superiore, il menu laterale e il punto in cui viene iniettata la pagina: `@@Body`.

```
@@inherits LayoutComponentBase

<MudThemeProvider Theme="@IcmTheme.Default" IsDarkMode="@@_isDarkMode" />
<MudPopoverProvider /> <MudDialogProvider /> <MudSnackbarProvider />

<MudLayout>
  <MudAppBar> ... nome utente + form POST /auth/logout ... </MudAppBar>
  <MudDrawer> <NavMenu /> </MudDrawer>           // menu a gruppi espandibili
  <MudMainContent>
    <IcmErrorBoundary> @@Body </IcmErrorBoundary> // ← QUI entra la pagina
  </MudMainContent>
</MudLayout>
```

Tre responsabilità del layout

- **Tema:** legge il claim `tema_preferito` dell'utente loggato (light/dark) al mount.
- **Provider:** Dialog/Snackbar/Popover devono esistere a monte di chi li usa → stanno qui.
- **Rete di sicurezza:** `IcmErrorBoundary` avvolge `@@Body` per catturare e loggare le eccezioni dei componenti senza far cadere il circuit.

NavMenu.razor è solo il contenuto del drawer: il link *Home* (→ `/dashboard`) e i tre gruppi espandibili (Tabelle Amm.ve, Attività studio, Reports). I link puntano alle pagine via `Href`.

6. Anagrafiche.razor — la pagina

È una pagina routable (`@@page "/anagrafiche"`). **Non parla mai col database:** inietta il **Manager** e i servizi UI, e da lì orchestra tutto.

```
@@page "/anagrafiche"
@@inject IAnagraficaManager AnagraficaManager // logica di business (mai il Repository!)
@@inject IDialogService DialogService         // per aprire i dialog
@@inject ISnackbar Snackbar                   // notifiche
@@inject IErrorLogger ErrorLogger              // log errori

protected override async Task OnInitializedAsync() => await RicaricaAsync();

private async Task RicaricaAsync() {
  _anagrafiche = await AnagraficaManager.ElencoAsync(); // UI → Manager → Repository → SQL
}
```

Il flusso dei dati (regola architetturale vincolante)

Anagrafiche.razor → IAnagraficaManager → IAnagraficaRepository → SQL Server . La UI **non** chiama mai direttamente il Repository: passa sempre dal Manager, che applica validazioni e regole di business.

La pagina gestisce: caricamento elenco in `MudDataGrid` , filtri, selezione riga (giallo), doppio-click (apre modifica), icona elimina per riga. Aggiungi/Modifica/Elimina **aprono dialog** via `DialogService` .

7. AnagraficaFormDialog.razor — il dialog

Componente condiviso (in **Components/Shared**) usato sia per *Aggiungi* che per *Modifica*: il parametro **Iniziale** discrimina i due flussi (null = nuovo, valorizzato = modifica).

```
// nella pagina: apertura del dialog passando l'anagrafica (o null)
var parameters = new DialogParameters<AnagraficaFormDialog> { { x => x.Iniziale, a } };
var dialog = await DialogService.ShowAsync<AnagraficaFormDialog>("Modifica anagrafica", parameters,
options);
var result = await dialog.Result;          // attende che l'utente confermi/annulli
if (result is { Canceled: false }) await RicaricaAsync(); // se salvato, ricarica
```

Dentro il dialog: un **MudForm** con i campi (i lookup Paese/Provincia arrivano da **ILookupRepository**), validazione, e su *Salva* chiama il Manager (**CreaAsync** / **AggiornaAsync**). Le eccezioni tipizzate (**AnagraficaInvalidaException**) diventano messaggi user-friendly; il dialog si chiude con **DialogResult.Ok(true)** .

8. La sequenza completa, end-to-end

1 **Avvio:** `dotnet run` → Program.cs registra DI + pipeline → `MapRazorComponents<App>()` → server in ascolto.



2 **Utente non loggato** apre `/` → fallback policy → redirect a `/login` .



3 **Login:** il form fa POST a `/auth/login` → `UtenteManager.AutenticaAsync` → cookie + claim → redirect.



4 **Richiesta pagina:** App.razor (HTML+script) → Routes.razor abbina l'URL → RouteView avvolge nel MainLayout.



5 **Render:** MainLayout disegna AppBar+Drawer+provider e inietta la pagina in `@@Body` .



6 **Dati:** Anagrafiche.razor → `AnagraficaManager.ElencoAsync()` → Repository (Dapper) → SQL → griglia.



7 **Modifica:** doppio-click → DialogService apre AnagraficaFormDialog → su Salva → Manager → Repository → SQL → ricarica.

9. Mappa delle relazioni

File	Ruolo	Chiama / contiene
Program.cs	Composizione: DI, pipeline, endpoint, seed	Registra tutto; <code>MapRazorComponents<App></code>
App.razor	Documento HTML radice + asset	Ospita <code><Routes></code> , carica <code>blazor.web.js</code>
Routes.razor	Router URL → pagina	Applica <code>MainLayout</code> di default
MainLayout.razor	Cornice + provider <code>MudBlazor</code>	Contiene <code>NavMenu</code> e <code>@Body</code>
NavMenu.razor	Menu laterale	Link <code>Href</code> alle pagine
Anagrafiche.razor	Pagina elenco/CRUD	Iniatta <code>IANagraficaManager</code> , apre il dialog
AnagraficaFormDialog.razor	Form Aggiungi/Modifica	Chiama <code>Crea/AggiornaAsync</code> , <code>ILookupRepository</code>

ICM Fatturazioni — documento tecnico interno. I riferimenti ai file puntano a `src/ICMFatturazioni.Web/` . Generato automaticamente a partire dal codice sorgente corrente.